



Projeto e Análise de Algoritmos

Complexidade de Espaço, Notação Assintótica e
Teorema Mestre

André Tavares da Silva

andre.silva@udesc.br

Complexidade de Espaço

Em alguns casos é importante considerarmos também a complexidade de espaço, i.e. o ‘espaço’ que é utilizado em função de ‘n’.

- Qual a complexidade dos algoritmos abaixo?

```
int n, f=1;
scanf("%i", &n);
for( ; n>0; n--)
    f = f * n;
return f;
```

```
int fatorial( int n ){
    if (n == 0)
        return 1;
    return n * fatorial( n-1 );
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
int fatorial( int n ){  
    int a, b, c, d, e, f, g;  
    if (n == 0)  
        return 1;  
    return n * fatorial( n-1 );  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo? **$O(n)$**

```
int fatorial( int n ){  
    int a, b, c, d, e, f, g;  
    if (n == 0)  
        return 1;  
    return n * fatorial( n-1 );  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
int fatorial( int n ){  
    int resultado = 1;  
    for (int i = 1; i <= n; i++)  
        resultado *= i;  
    return resultado;  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo? **$O(1)$**

```
int fatorial( int n ){  
    int resultado = 1;  
    for (int i = 1; i <= n; i++)  
        resultado *= i;  
    return resultado;  
}
```

Complexidade de Espaço

- E deste algoritmo?

```
int pesqbin(int *v, int p, int r, int e){  
    int q;  
    if ( r < p )  
        return -1;  
    q = (p + r)/2;  
    if (e == v[q])  
        return q;  
    if (e < v[q])  
        return pesqbin(v, p, q-1, e);  
    else  
        return pesqbin(v, q+1, r, e);  
}
```

Complexidade de Espaço

- E deste algoritmo? **$O(\log n)$**

```
int pesqbin(int *v, int p, int r, int e){
    int q;
    if ( r < p )
        return -1;
    q = (p + r)/2;
    if (e == v[q])
        return q;
    if (e < v[q])
        return pesqbin(v, p, q-1, e);
    else
        return pesqbin(v, q+1, r, e);
}
```


Complexidade de Espaço

- E deste outro?

```
int foo( int n ){  
    int v[n], i, s = 0;  
    if ( n==0 ) return 0;  
    for( i=0; i<n; i++ )  
        v[i] = i*2;  
    return foo( n-1 );  
}
```

Complexidade de Espaço

- E deste outro? **$O(n)$**

```
int foo( int n ){  
    int v[n], i, s = 0;  
    if ( n==0 ) return 0;  
    for( i=0; i<n; i++ )  
        v[i] = i*2;  
    return  foo( n-1 );  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
int **alocaMatriz(int n) {  
    int **matriz = new int*[linhas];  
    for (int i = 0; i < linhas; i++)  
        matriz[i] = new int[colunas];  
    return matriz;  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo? **$O(n^2)$**

```
int **alocaMatriz(int n) {  
    int **matriz = new int*[linhas];  
    for (int i = 0; i < linhas; i++)  
        matriz[i] = new int[colunas];  
    return matriz;  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
int fibonacci ( int n ){  
    int i, fib[n+1];  
    if( n<=1 )  
        return 1;  
    fib[0] = fib[1] = 1;  
    for( i=2; i<=n; i++ )  
        fib[i] = fib[i-1] + fib[i-2];  
    return fib[n];  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo? **$O(n)$**

```
int fibonacci ( int n ){  
    int i, fib[n+1];  
    if( n<=1 )  
        return 1;  
    fib[0] = fib[1] = 1;  
    for( i=2; i<=n; i++ )  
        fib[i] = fib[i-1] + fib[i-2];  
    return fib[n];  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
int fib( int n ){  
    if( n == 0 || n == 1 )  
        return 1;  
    return fib( n-1 ) + fib( n-2 );  
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo? $O(2^n)$

```
int fib( int n ){  
    if( n == 0 || n == 1 )  
        return 1;  
    return fib( n-1 ) + fib( n-2 );  
}
```


Lembrando: Recursividade de cauda

- Uma função apresenta recursividade de cauda se nenhuma operação é executada após o retorno da chamada recursiva, exceto retornar seu valor. Ou seja, a chamada recursiva é o último comando da função.
- Em geral, compiladores, que executam otimizações de código, substituem as funções que apresentam recursividade de cauda por uma versão não recursiva dessa função.

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

```
void bubble(int *v, int n){
    int i, j, aux;
    for (i = n - 1; i > 0; i--){
        for (j = 0; j < i; j++){
            if (v[j] > v[j+1]){
                aux = v[j];
                v[j] = v[j+1];
                v[j+1] = aux;
            }
        }
    }
}
```

Complexidade de Espaço

- E qual a complexidade de espaço deste algoritmo?

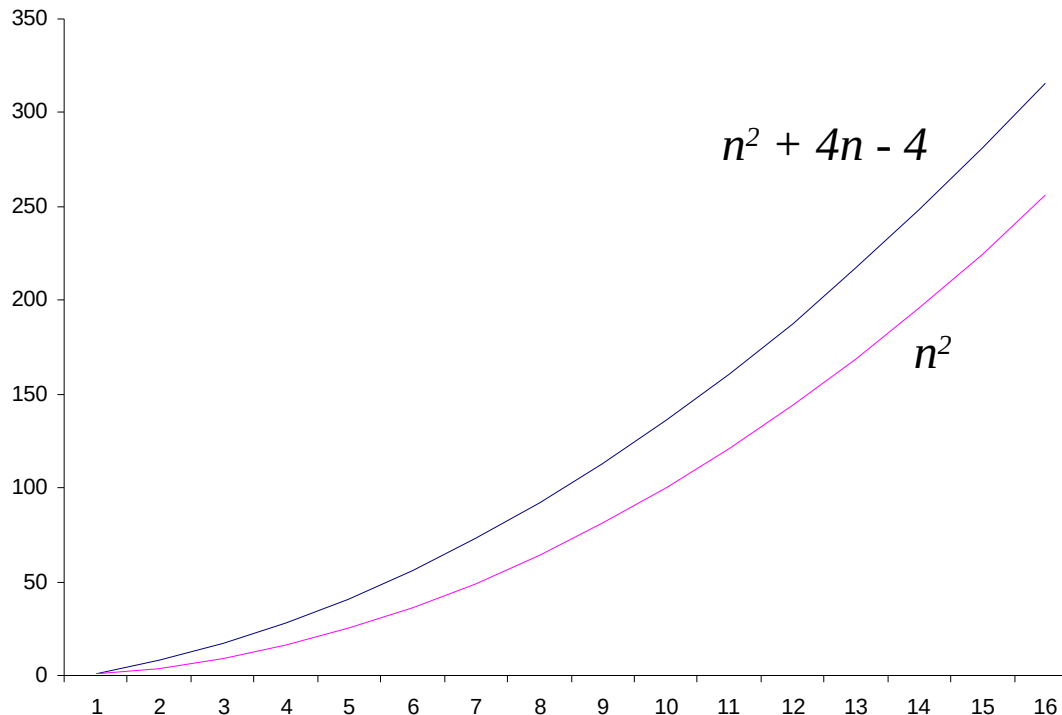
```
void bubble(int *v, int n){
    int i, j, aux;
    for (i = n - 1; i > 0; i--){
        for (j = 0; j < i; j++){
            if (v[j] > v[j+1]){
                aux = v[j];
                v[j] = v[j+1];
                v[j+1] = aux;
            }
        }
    }
}
```

$O(1)$, pois só usa o vetor que já existia antes.

Notação Assintótica

Limite Inferior (Notação Ω)

Uma função $f(n)$ é o limite inferior de outra função $g(n)$ se existem duas constantes positivas c e n_0 tais que, para $n > n_0$, temos $|g(n)| \geq c \cdot |f(n)|$, $g(n) = \Omega(f(n))$.

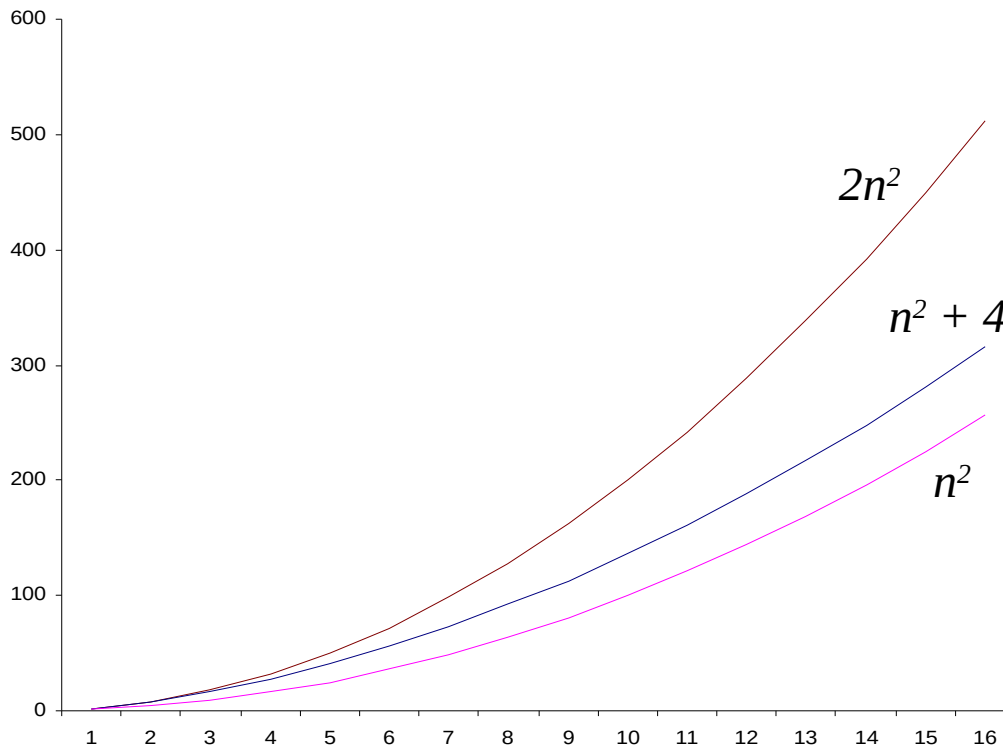


$$n^2 + 4n - 4 = \Omega(n^2)$$

Notação Assintótica

Limite Firme (Notação Θ)

Uma função $f(n)$ é o limite restrito (ou exato) de outra função $g(n)$ se existem três constantes positivas c_1 , c_2 , e n_0 tais que, para $n > n_0$, temos $c_1 \cdot |f(n)| \geq |g(n)| \geq c_2 \cdot |f(n)|$, $g(n) = \Theta(f(n))$



[funções crescem
com mesma rapidez]

$$n^2 + 4n - 4 = \Theta(n^2)$$

Notação Assintótica

Agora considere as funções $f(x) = n^{\epsilon}$ (onde $\epsilon > 0$ e $\epsilon < 1$) e $g(x) = \log n$.

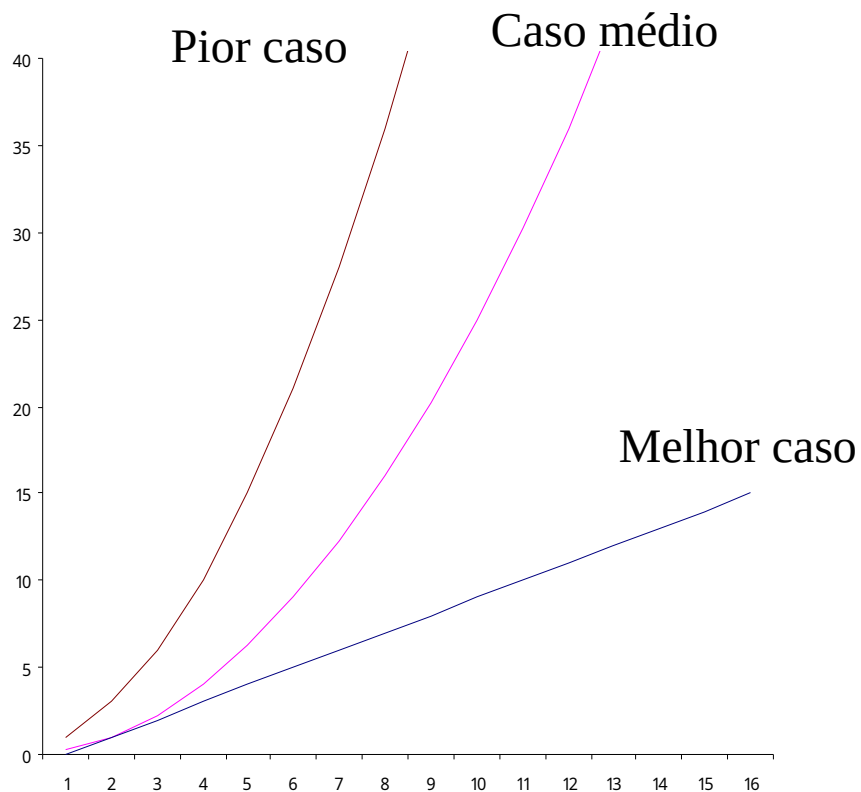
Qual a relação entre $f(x)$ e $g(x)$?

$$f(x) = O(g(x))$$

$$f(x) = \Omega(g(x))$$

$$f(x) = \Theta(g(x))$$

Ordenação por Inserção



Pior Caso: $O(n^2)$

Caso Médio: $n^2/3$

Melhor Caso: $\Omega(n)$.

Teorema Mestre

O Teorema Mestre é um método para determinar a complexidade de algoritmos recursivos que se encaixam na forma $T(n) = aT(n/b) + f(n)$, onde ' a ' é o número de subproblemas, ' b ' é o fator de divisão e $f(n)$ é o custo de outras operações.

A complexidade é encontrada comparando $f(n)$ com $n^{\log_b a}$ e dividido em três casos distintos que serão vistos a seguir.

Teorema Mestre

Livro de receitas para resolver recorrências da forma:

$$T(n) = a T(n / b) + f(n)$$

peso do número de chamadas recursivas
vs.
peso de cada chamada recursiva individual

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ definida sobre os inteiros não negativos pela recorrência acima, então $T(n)$ pode ser limitado assintoticamente como:

Teorema Mestre

Caso 1:

Se

$$f(n) = \Theta(n^{\log_b a})$$

então:

$$T(n) = \Theta(n^{\log_b a} \times \log n)$$

Exemplo: $T(n) = T(2n / 3) + 1$

Teorema Mestre

Caso 2:

Se

$$f(n) = O(n^{\log_b a - \epsilon})$$

para alguma constante $\epsilon > 0$, então:

$$T(n) = \Theta(n^{\log_b a})$$

Exemplo: $T(n) = 9T(n / 3) + n$

Teorema Mestre

Caso 3:

Se

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

para alguma constante $\epsilon > 0$, e se
 $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e para
todo n suficientemente grande, então:

$$T(n) = \Theta(f(n))$$

Exemplo: $T(n) = 3T(n/4) + n \log n$

Teorema Mestre

Casos especiais:

Se as condições do caso 1, 2 ou 3 não forem satisfeitas então não é possível resolver a recorrência usando o teorema mestre!

Exemplo: $T(n) = 2T(n / 2) + n \log n$

Atividade

Considere o algoritmo abaixo e descreva a sua relação de recorrência, sua complexidade de tempo e espaço para o melhor e pior caso:

```
int pow(int base, int exp) {  
    if( exp==0 ) return 1;  
    int ret = pow( base, exp/2 );  
    ret = ret * ret;  
    if( exp%2 == 1 ) ret = ret * base;  
    return ret;  
}
```

Atividade

Considere o algoritmo abaixo e descreva a sua relação de recorrência, sua complexidade de tempo e espaço para o melhor e pior caso:

```
double foo( int* v, int n, int p ){  
    if( n<=0 )  
        return 0;  
    int i, soma = 0;  
    for( int i=0; i<n ; i=i+p )  
        soma += v[i];  
    return sqrt(soma) + foo( v, soma%n, p );  
}
```

Quais são os possíveis resultados para uma operação de resto de divisão?
(Pense para um número fixo – ex: $x\%7$)

Atividade

Use o método mestre para fornecer limites assintóticos restritos para as seguintes recorrências:

a. $T(n) = T(n/2) + 1$

b. $T(n) = 4T(n/2) + n$

c. $T(n) = 4T(n/2) + n^3$